

EHR-RAG: Enhanced Retrieval-Augmented Generation for Long-Horizon Structured Electronic Health Records

*Technical Writeup — Design, Architecture, and
Production Integration for On-Premises Clinical RAG Systems*

December 13, 2025

Abstract

Electronic Health Records (EHRs) encode rich longitudinal evidence spanning years of clinical events, diagnoses, medications, and laboratory results. Their sheer volume consistently exceeds the fixed context windows of large language models (LLMs), forcing practitioners to choose between truncation—which discards critical temporal evidence—or sophisticated retrieval strategies that preserve clinical structure. This writeup presents **EHR-RAG** [1], a retrieval-augmented framework that addresses this challenge through three synergistic components: Event- and Time-Aware Hybrid EHR Retrieval (ETHER), Adaptive Iterative Retrieval (AIR), and Dual-Path Evidence Retrieval and Reasoning (DER). We bridge EHR-RAG’s academic design to a fully on-premises, HIPAA-conscious production system built on FastAPI [12], PostgreSQL, ChromaDB [11], and Ollama running **Mistral 7B** [8] locally—with all inference remaining on-device and no data leaving the network. The experimental results reported in Section 4 are drawn directly from the original EHR-RAG paper [1], which used frontier cloud models (GPT-5 and Claude-Opus-4.5) as its backbones; our local system uses Mistral 7B and does not replicate those numbers. The purpose of this writeup is to document the framework’s design and its practical adaptation to an air-gapped clinical environment.

1 Introduction

Structured EHR data differ fundamentally from the free-text documents that most RAG systems are designed to handle. Each patient record is a chronologically ordered sequence of typed clinical events $e_i = (c_i, v_i, \tau_i)$, where c_i is a clinical concept (diagnosis, procedure, lab test, medication), v_i is the observed value (numeric or categorical), and τ_i is the event timestamp. In long-horizon care settings, a single patient’s record may span decades and contain thousands of heterogeneous events—far beyond the practical context limits of any current LLM [5].

Naive approaches to this problem fall into two camps. **Truncation** retains only the most recent events up to the context window, discarding early disease onset signals that are often the strongest predictors

of chronic outcomes. **Vanilla RAG** retrieves event subsets by semantic similarity, but dense text embeddings are insensitive to numerical magnitude and temporal ordering—two dimensions that are diagnostically critical in clinical data.

EHR-RAG [1] is specifically designed to overcome these limitations. Rather than treating retrieval as a single text-similarity lookup, it models three distinct axes of clinical relevance: *event type* (numeric vs. textual), *temporal position* (recency vs. onset), and *hypothesis polarity* (evidence for vs. against the target outcome).

Honest scope of this writeup. The performance numbers in Section 4 are reproduced from the original EHR-RAG paper [1] and reflect experiments run with GPT-5 and Claude-Opus-4.5—large frontier models. The local production system described in Section 3 uses **Mistral 7B** [8] via Ollama [13] as its only inference model. Mistral 7B is a 7-billion-parameter open-weights model; its reasoning depth is meaningfully lower than frontier models, particularly on multi-step clinical prediction tasks. We do not claim that the production system achieves EHRSHOT-level accuracy. The value of the local system is its complete data locality and PHI containment—not its raw benchmark scores.

This writeup proceeds as follows. Section 2 details the three core components of EHR-RAG. Section 3 maps these components onto the local on-premises production architecture. Section 4 summarises experimental results from the original paper. Section 5 closes with design implications.

2 EHR-RAG Framework Architecture

2.1 Task Formulation

Given a patient p with longitudinal EHR $E_p = \{e_i\}_{i=1}^N$, ordered chronologically ($\tau_1 \leq \tau_2 \leq \dots \leq \tau_N$), the objective is to predict a clinical outcome

$y \in \mathcal{Y}$ at a specified prediction time τ^* using only the history available up to that point:

$$E_p^{\leq \tau^*} = \{e_i \in E_p \mid \tau_i \leq \tau^*\}.$$

Since directly feeding $E_p^{\leq \tau^*}$ to an LLM $\mathcal{M}$ is infeasible when N is large, EHR-RAG constructs a compact, task-relevant evidence context $\mathcal{C} \subset E_p^{\leq \tau^*}$ such that $\hat{y} = \mathcal{M}(\mathcal{C})$ approximates the ground-truth outcome y as closely as possible.

2.2 ETHER: Event- and Time-Aware Hybrid Retrieval

Structured EHRs contain two fundamentally different event modalities. **Numeric events** (lab values, vital signs) require magnitude-aware retrieval; dense embeddings computed from text representations of numbers are known to be insensitive to numerical differences, making semantic similarity alone unreliable for this modality. **Textual events** (diagnoses, procedure descriptions, clinical notes) are well-suited to embedding-based retrieval but benefit from temporal weighting.

ETHER addresses these dual modalities via a two-track pipeline:

Indicator-wise numeric aggregation. All numeric events are grouped by clinical indicator k (e.g., “serum creatinine”), producing per-indicator value trajectories $V_k = \{(v_{k,j}, \tau_{k,j}) \mid \tau_{k,j} \leq \tau^*\}$. Coarse retrieval over indicator-name embeddings yields a candidate indicator set K_{coarse} ; LLM-based reranking then selects a fine-grained subset $K_{\text{fine}} \subset K_{\text{coarse}}$. For each retained indicator, the N_{recent} most recent measurements are included in the numeric evidence set E_{num} .

U-shaped time-aware textual retrieval. Non-numeric events are chunked and embedded in a vector store. Given a task query q_{task} , candidate chunks are scored by a convex combination of semantic similarity s_{sem} and a U-shaped temporal relevance function:

$$s_{\text{time}}(\tau_c) = \max\left(e^{-\frac{\tau^* - \tau_c}{\tau_{\text{recent}}}}, e^{-\frac{\tau_c - \tau_{\text{first}}}{\tau_{\text{early}}}}\right),$$

$$s(c) = \alpha s_{\text{sem}}(q_{\text{task}}, c) + (1 - \alpha) s_{\text{time}}(\tau_c).$$

The U-shape is clinically motivated: both the most recent events (acute changes) and the earliest events (disease onset, initial presentation) carry disproportionate predictive weight, while mid-history events are generally less informative. Top- K_{final} chunks, temporally reordered, constitute E_{text} . The final ETHER evidence is $E_{\text{ETHER}} = E_{\text{num}} \cup E_{\text{text}}$.

2.3 AIR: Adaptive Iterative Retrieval

Single-pass retrieval is insufficient when clinically relevant evidence is temporally dispersed or only indirectly related to the initial query. AIR operationalises an iterative evidence expansion loop, conceptually related to the ReAct framework [6] but constrained to a single clinical dimension per iteration. Starting from $q^{(0)} = q_{\text{task}}$ and initial evidence $E^{(0)}$, at each iteration t the LLM assesses sufficiency. If insufficient, it generates a refined query targeting one missing clinical aspect:

$$q^{(t+1)} = \mathcal{M}_R(q^{(t)}, E^{(t)}),$$

$$E^{(t+1)} = \text{Merge}\left(E^{(t)}, \text{Retrieve}(q^{(t+1)})\right),$$

where $\text{Merge}(\cdot)$ performs deduplication and temporal reordering. Iteration continues until the LLM declares sufficiency or a predefined iteration limit is reached.

2.4 DER: Dual-Path Evidence Retrieval and Reasoning

Clinical prediction involves competing hypotheses, and single-path reasoning is susceptible to confirmation bias [15]. DER constructs two complementary queries: a *factual* query q^+ seeking evidence supporting the target outcome, and a *counterfactual* query q^- seeking evidence against it. Each path runs independently through AIR, producing evidence sets E_{text}^+ and E_{text}^- . The LLM then forms explicit hypotheses:

$$h^+ = \mathcal{M}(E_{\text{text}}^+ \cup E_{\text{num}}), \quad h^- = \mathcal{M}(E_{\text{text}}^- \cup E_{\text{num}}).$$

Evidence is fused: $E_{\text{fuse}} = E_{\text{text}}^+ \cup E_{\text{text}}^- \cup E_{\text{num}}$. The final prediction requires the LLM to compare the strength, directness, and clinical relevance of each hypothesis:

$$\hat{y} = \mathcal{M}_{\text{dec}}(E_{\text{fuse}}, h^+, h^-).$$

This dual-path design mitigates spurious correlations and substantially improves robustness on class-imbalanced clinical prediction tasks.

3 Production Implementation: On-Premises Medical RAG System

The local production system implements a strict on-premises constraint: no data or prompts leave the local network. PHI (Protected Health Information) is present throughout, so the architecture is designed from the ground up for HIPAA compliance, with every component containerised via Docker Compose.

3.1 Core Infrastructure

PostgreSQL 16 serves as the authoritative structured store for all tabular clinical and billing data. It hosts the `source_catalog` table that tracks every ingested data stream, its column signature, and all associated metadata. MIMIC-III [3] clinical tables and billing/RCM tables are stored in separate schemas, never shared, in adherence to a strict no-cross-contamination policy.

ChromaDB [11] stores dense vector embeddings exclusively for *unstructured* documents (clinical PDFs, discharge summaries, guidelines). The textual retrieval arm of ETHER maps directly to ChromaDB-backed similarity search.

Ollama + Mistral 7B runs `nomic-embed-text` [9] for 8192-token embedding generation and **Mistral 7B** [8] for all text generation—both entirely on the host machine via Ollama [13]. No API call leaves the network perimeter. Mistral 7B was chosen for its favourable balance of instruction-following quality and inference speed on consumer hardware, but it is a 7-billion-parameter model and does not match the reasoning depth of GPT-5 or Claude-Opus-4.5. Queries that require multi-step clinical reasoning may produce shorter or less nuanced responses than what is reported in the paper’s benchmark results.

FastAPI [12] exposes the RAG pipeline at port 8080 with a Streamlit-based frontend at port 8501. All admin operations are restricted to a CLI tool (`catalog_admin.py`) invoked via `docker exec`—no unauthenticated HTTP admin surface.

3.2 Structure-Defined Streams and the Catalog

A central design challenge in deploying EHR-RAG over real-world tabular data is schema heterogeneity: billing files, lab exports, and demographic CSVs arrive in ad-hoc formats with inconsistent column naming. The local system resolves this via **structure-defined streams**: every incoming CSV or Excel file is identified by a 32-character MD5 signature computed from its sorted, sanitised column headers—not its filename. This signature is the stream identity.

The ingestion pipeline operates in a two-pass column-processing model. *Pass 1* computes the pre-denylist signature from all safe identifiers. *Pass 2* applies a PHI denylist (covering SSN variants, passwords, and any tokens the administrator adds) to determine which columns are physically stored, without altering the computed signature. This guarantees

that a file with the same schema always routes to the same staging table even after PHI columns have been stripped.

Each stream lands in a PostgreSQL staging table `stg_sig8`. Administrators expose a stream for text-to-SQL querying by running:

```
docker exec medical_rag_backend \
  python scripts/catalog_admin.py expose <
  sig8>
```

This creates a PostgreSQL view `v_LABEL` using safe cast helpers (`safe_numeric`, `safe_date`, `safe_timestampz`)—all defined as IMMUTABLE PL/pgSQL functions that return NULL on any bad input rather than raising an exception. Downstream text-to-SQL queries target only these views, never the raw staging tables, providing a clean semantic interface analogous to EHR-RAG’s “concept verbalization” step.

3.3 Text-to-SQL and Schema Linking

Structured queries over the exposed views are handled by Mistral 7B [8] via a text-to-SQL pipeline. The critical component is schema linking: the model is presented only with the tables and columns relevant to the user query, not the entire schema. A token-overlap scoring function ranks views by how many user-query tokens appear in either the view’s human label or its canonical column names. The top-*N* views are passed to Mistral 7B alongside few-shot SQL examples.

Generated SQL is validated through a `sqlglot` [14] AST parse before execution, verifying it is a single `SELECT` statement with no DML or DDL. Grounding failures—where the generated answer is not supported by retrieved evidence—trigger an explicit refusal message, not a hallucinated response. A CrossEncoder reranker [10] re-scores retrieved chunks before they are presented to the model.

3.4 HIPAA Audit Logging

All query events are written to a JSON-lines rotating audit log with one record per query. Each record captures: timestamp, user session, query text (not patient data), tables accessed, and retrieval scores. No PHI appears in the audit log; patient data remains exclusively in PostgreSQL.

3.5 Mapping EHR-RAG Components to the Local System

Table 1 maps the three EHR-RAG components to their local implementation counterparts.

Table 1: EHR-RAG components mapped to local system.

EHR-RAG Component		Local Implementation
ETHER track	numeric	PostgreSQL views with <code>safe.numeric</code> casts; indicator-level staging tables per stream signature
ETHER track	textual	ChromaDB dense retrieval [11] with U-shape temporal scoring via ingestion timestamps
AIR iterative retrieval	re-	Multi-turn query orchestration in FastAPI; CrossEncoder [10] reranker re-scores after each pass
DER dual-path reasoning	re-	Dual-query compound orchestration with hallucination checker and relevance grader; refusal on grounding failure
Concept verbalization	re-	Human-labelled views (<code>v.LABEL</code>); schema-link scoring at query time

4 Experimental Results and Ablation

Important note. All results in this section are reproduced from the original EHR-RAG paper [1]. The experiments were conducted by the paper’s research team using GPT-5 and Claude-Opus-4.5 as LLM backbones against the EHRSHOT benchmark [2]. The local on-premises system described in Section 3 uses **Mistral 7B** and has not been independently evaluated against EHRSHOT. The numbers below illustrate the framework’s *design value*—they should not be interpreted as locally reproducible without a comparably large LLM.

4.1 Benchmark and Tasks

EHR-RAG is evaluated on the EHRSHOT benchmark [2], which captures longitudinal patient records across general hospital care. Compared to MIMIC-IV [4], EHRSHOT contains 2.3× more clinical events and 95.2× more encounters per patient—making it substantially more challenging for long-horizon reasoning. Four prediction tasks are evaluated: **Long Length of Stay** (binary), **30-Day Readmission** (binary), **Acute Myocardial Infarction** (binary), and **Anemia** (four-class). All tasks are class-imbalanced.

4.2 Main Results (Paper, GPT-5 Backbone)

Table 2 summarises performance against LLM-based baselines as reported in the original paper [1].

A notable finding is that *vanilla RAG underperforms*

Table 2: Macro-F1 (%) on four EHRSHOT tasks. Results from [1] using GPT-5. Δ : gain over strongest baseline.

Method	LLoS	Readm.	Acute MI	Anemia
Direct Generation	66.52	44.37	57.97	28.38
Vanilla RAG [5]	64.77	43.40	59.83	20.44
Uniform RAG	61.50	49.63	47.15	32.39
Rule-based RAG	56.58	44.08	52.06	26.21
ReAct RAG [6]	60.85	45.22	56.49	31.64
EHR-RAG [1]	70.15	60.91	76.29	44.07
Gain (Δ)	+3.63	+11.28	+16.46	+11.68

direct generation on Long Length of Stay (64.77 vs. 66.52 Macro-F1). Without temporal weighting, semantic similarity retrieval tends to surface clinically stale events, omitting recent acute signals. ETHER’s U-shape retrieval directly corrects this pathology.

4.3 Ablation: Component Contributions

Table 3 shows the contribution of each component on Long Length of Stay and Anemia, from [1].

Table 3: Ablation results from [1]. Macro-F1 (%), GPT-5.

Variant	LLoS		Anemia	
	Acc	F1	Acc	F1
EHR-RAG (full)	74.12	70.15	57.01	44.07
w/o ETHER	72.94	68.47	53.39	40.12
w/o AIR	72.16	68.19	54.07	41.44
w/o DER	70.59	67.55	46.83	38.58
Vanilla RAG	68.24	64.77	42.08	20.44

Removing DER causes the largest Macro-F1 drop on Anemia (−5.49 points), confirming that dual-path hypothesis evaluation is most critical for multi-class, imbalanced tasks where confirmation bias is most dangerous. Removing AIR causes a uniform ~2-point decline, and removing ETHER shows the largest accuracy degradation on Anemia (−3.62 points).

4.4 Cross-Architecture Generalisation

The paper evaluates across GPT-5, Claude-Opus-4.5, and LLaMA-3.1-8B [1]. Consistent Macro-F1 gains appear across all three backbones, with the largest relative improvement on the smaller LLaMA-3.1-8B—suggesting EHR-RAG’s scaffolding partially compensates for limited model capacity. This finding is particularly relevant to the local deployment: Mistral 7B [8] is broadly comparable in scale to LLaMA-3.1-8B, and the framework’s design—explicit temporal scoring, iterative retrieval, dual-path reasoning—is

precisely the kind of structured scaffolding that helps smaller models perform beyond their zero-shot baseline. We expect the local system to see similar *relative* gains from the framework’s design, even if the absolute accuracy remains below frontier-model results.

4.5 Low-Resource Performance

Against ML baselines (count-based and CLMBR-based logistic regression [2]), EHR-RAG’s zero-shot approach outperforms both in low-resource regimes (≤ 16 labeled samples per class) across all tasks. This data-efficiency property is directly relevant in billing and RCM settings where labeled outcome data is scarce.

5 Conclusion

EHR-RAG demonstrates that the dominant limitations of LLM-based clinical prediction—context overflow, temporal blindness, and confirmation bias—can be addressed through principled retrieval design rather than through larger models or more training data [1]. Its three components (ETHER, AIR, DER) are mutually reinforcing: ETHER curates high-quality, type-aware initial evidence; AIR iteratively fills evidence gaps; DER guards against single-path bias in the final inference step.

For the local on-premises system, EHR-RAG’s architecture provides a concrete blueprint for structured retrieval over heterogeneous clinical and billing data. The structure-defined stream model maps cleanly to ETHER’s separation of numeric and textual modalities. The schema-linking and text-to-SQL pipeline maps to ETHER’s concept verbalization. The CrossEncoder reranker [10] and compound query orchestration align with AIR’s iterative refinement. The hallucination checker and grounding refusal mechanism align with DER’s dual-hypothesis validation.

Honest limitations of the local deployment.

The local system runs Mistral 7B [8]—a capable but significantly smaller model than the GPT-5 backbone used in the paper. Complex multi-hop clinical reasoning, nuanced counterfactual evaluation, and low-shot generalisation over rare events will all be weaker than the benchmark results suggest. For a billing and RCM use case—pattern retrieval over structured claims, SQL queries over structured billing views, and document lookup over clinical PDFs—Mistral 7B is adequate. For primary clinical decision support at frontier accuracy levels, a larger model would be required, and the strict no-external-API constraint would need to be reconsidered or addressed through

a privately hosted model at larger scale (e.g., Mistral-Large or a self-hosted 70B model).

The most important practical lesson from EHR-RAG is that clinical data’s temporal structure must be *modelled explicitly*. U-shaped temporal scoring, indicator-level numeric aggregation, and iterative evidence expansion each encode clinical intuitions that translate into measurable gains—and that translate equally into design decisions that make a production medical RAG system trustworthy enough to operate over real PHI, regardless of which model sits at the center.

Security Note. All inference is performed by on-premises Ollama models. No patient data, prompts, or generated text traverses any external API. The MIMIC-III [3] and billing/RCM schemas are physically isolated and are never joined except on verified shared keys.

References

References

- [1] EHR-RAG: Bridging Long-Horizon Structured Electronic Health Records and Large Language Models via Enhanced Retrieval-Augmented Generation. *arXiv preprint arXiv:2601.21340*, January 2026. (Primary source for the EHR-RAG framework, all experimental results, and EHRSHOT benchmark comparisons in this writeup.)
- [2] M. Wornow, Y. Xu, R. Thapa, B. Patel, C. Arnaout, I. S. Steinberg, J. A. Fries, and N. H. Shah. The shaky foundations of clinical foundation models: A survey of large language models and foundation models for EHRs. *npj Digital Medicine*, 2023.
- [3] A. E. W. Johnson, T. J. Pollard, L. Shen, L.-W. H. Lehman, M. Feng, M. Ghassemi, B. Moody, P. Szolovits, L. Anthony Celi, and R. G. Mark. MIMIC-III, a freely accessible critical care database. *Scientific Data*, 3:160035, 2016.
- [4] A. E. W. Johnson, L. Bulgarelli, L. Shen, A. Gayles, A. Shammout, S. Horng, T. J. Pollard, S. Hao, B. Moody, B. Gow, et al. MIMIC-IV, a freely accessible electronic health record dataset. *Scientific Data*, 10(1):1, 2023.
- [5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

- [7] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511*, 2023.
- [8] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023. (*LLM used for all local text generation in the production system.*)
- [9] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar. Nomic Embed: Training a reproducible long context text embedder. *arXiv preprint arXiv:2402.01613*, 2024. (*nomic-embed-text model used for all local vector embeddings.*)
- [10] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP*, 2019. (*CrossEncoder reranker used for relevance scoring in the local retrieval pipeline.*)
- [11] Chroma. ChromaDB: The AI-native open-source embedding database. <https://www.trychroma.com>, 2023. (*Vector store for unstructured document embeddings in the local system.*)
- [12] S. Ramírez. FastAPI: Modern, fast web framework for building APIs with Python 3.7+. <https://fastapi.tiangolo.com>, 2018–present. (*Backend API framework for the local RAG system.*)
- [13] Ollama. Ollama: Get up and running with large language models locally. <https://ollama.com>, 2023. (*Local inference runtime for Mistral 7B and nomic-embed-text.*)
- [14] T. Abrams. sqlglot: A Python SQL parser, transpiler, and optimizer. <https://github.com/tobymao/sqlglot>, 2021–present. (*AST-level SQL validation used to block DML/DDI injection in the text-to-SQL pipeline.*)
- [15] M. L. Graber, N. Franklin, and R. Gordon. Diagnostic error in internal medicine. *Archives of Internal Medicine*, 165(13):1493–1499, 2005. (*Motivation for DER’s dual-path design to counter confirmation bias.*)